EDUCAMENTE AI

Como funciona a Inteligência Artificial

Documentação técnica e pedagógica do agente de IA da plataforma

Documento de referência · novembro de 2026
Baseado no código do serviço `agent/` e no Business Model Canvas

Sumário

1. Visão geral: o que a IA faz e o que não faz
2. Onde a IA se encaixa no modelo de negócio
3. Arquitetura do serviço
4. O agente: como ele trabalha
5. As oito ferramentas do agente
6. Fluxo 1 — Análise de aprendizagem
7. Fluxo 2 — Geração de atividade personalizada
8. Referência do fluxo: entradas, saídas e lógica
9. Progresso em tempo real e tela de espera
10. Cálculos determinísticos (o que o sistema calcula sem IA)
11. Proteções pedagógicas e de linguagem
12. Privacidade e proteção de dados (LGPD)
13. Segurança e controle de acesso
14. Provedores de IA e troca de provedor
15. Tratamento de erros
16. Modo demonstração
17. Testes e qualidade
18. Custos de IA e indicadores
19. Limitações conhecidas e próximos passos

1. Visão geral: o que a IA faz e o que não faz

A EducaMente AI usa um **agente de Inteligência Artificial** para transformar os resultados das avaliações registradas pelos professores em duas entregas:

- **Análise de aprendizagem da turma**: resumo, pontos de atenção e recomendações, cada recomendação ligada à evidência numérica que a sustenta.
- **Atividade personalizada**: título, instruções, exemplos resolvidos, questões, gabarito, orientação ao professor e justificativa da personalização, sempre salva como **rascunho** para o professor revisar.

> **Princípio central:** o agente não é um chatbot. Ele trabalha com **ferramentas reais** no servidor (consultas ao banco, cálculos, geração, validação e gravação). Os números são calculados pelo sistema; a IA só interpreta. E toda decisão pedagógica final continua com o professor.

A IA faz

- Consulta avaliações e resultados da turma.
- Interpreta indicadores calculados pelo sistema.
- Escreve resumos e recomendações com evidência.
- Planeja uma intervenção e gera a atividade.
- Revisa a própria atividade antes de entregar.

A IA não faz

- Não calcula médias nem percentuais.
- Não aprova atividades (só o professor aprova).
- Não faz diagnósticos nem usa rótulos sobre estudantes.
- Não recomenda reprovação ou retenção.

- Não vê nomes de estudantes nem observações do professor.

2. Onde a IA se encaixa no modelo de negócio

No modelo B2G descrito no Business Model Canvas, a Prefeitura/Secretaria Municipal de Educação contrata a plataforma e o professor é o usuário principal. A IA cobre os passos 4, 5 e 6 do "exemplo de funcionamento":

Etapa do Canvas	Como a IA atende
4. O agente consulta os dados e identifica conteúdos que precisam de atenção	Ferramentas `buscar_resultados` e `analisar_desempenho`; conteúdos abaixo de 70% são destacados.
5. A plataforma apresenta análises e sugestões de atividades	Fluxo de análise (registrada no histórico) e fluxo de geração (rascunho de atividade).
6. O professor revisa e decide	Atividades nascem como `draft`; editar volta para rascunho; aprovar é uma rota que só o professor chama.
Recomendação 3 — "Não depender exclusivamente da IA"	Médias, percentuais, distribuição e evolução são calculados em código (determinístico).
Recomendação 4 — "Manter o professor no controle"	O nível, o público-alvo, o tipo e a quantidade de questões são escolhas do professor; o agente é obrigado a respeitá-las.
Recomendação 5 — "Proteção de dados como prioridade"	Pseudônimos (E1, E2...), sem nomes nem observações para o provedor de IA, acesso via RLS.

3. Arquitetura do serviço

O agente é um serviço separado em **Next.js (somente API, Route Handlers) com TypeScript**, na pasta `agent/`. O front-end estático (`index.html`, `dashboard.html`) chama esse serviço enviando o token da sessão do professor no Supabase.

Front-end (navegador)
Painel do professor. Guarda os nomes dos estudantes e traduz os códigos E1, E2... de volta para nomes localmente.

Rotas da API — `agent/app/api/...`
Validação de entrada (Zod), autenticação, CORS, limite de uso e conversão de erros em respostas claras.

Autenticação — `src/lib/auth.ts`
Valida o token e cria um cliente Supabase que consulta o banco como o professor (as regras RLS continuam valendo).

Agente — `src/lib/agent/`
Prompts, ferramentas, loop do agente e as duas tarefas (análise e geração).

Domínio — `src/lib/domain/`
Cálculos de desempenho, pseudônimos, níveis, plano de intervenção, validação estrutural e proteções de linguagem.

Dados — `src/lib/data/`
Repositório Supabase (produção) e repositório em memória (testes e modo demonstração).

Provedor de IA — `src/lib/ai/`
Interface neutra (`AiProvider`) com adaptadores para Anthropic (Claude) e Google Gemini.

Rotas da API

Rota	O que faz
`POST /api/agent/analyze`	Análise de aprendizagem de uma turma (filtros opcionais: avaliação, componente, conteúdo, período). Grava em `pedagogical_analyses`.
`POST /api/agent/activities`	Gera uma atividade e salva com `status = "draft"` e `origin = "ai"`.
`GET /api/agent/history?class_id=`	Histórico de análises e atividades geradas com IA da turma.
`PATCH /api/activities/:id`	Edição pelo professor; a atividade volta a ser rascunho.
`POST /api/activities/:id/approve`	Aprovação pelo professor. O agente nunca chama esta rota.
`GET /api/health`	Mostra se Supabase e chave de IA estão configurados, sem expor segredos.
`POST /api/demo/...`	Rotas do modo demonstração (seção 16), desligadas por padrão.

Todas as rotas principais exigem `Authorization: Bearer <token da sessão Supabase>`. As rotas de análise e geração também aceitam `Accept: application/x-ndjson` para transmitir o progresso em tempo real (seção 9).

4. O agente: como ele trabalha

O agente segue um **loop de ferramentas** (`run-agent.ts`): o modelo de IA pede uma ferramenta, o servidor a executa de verdade, devolve o resultado ao modelo e repete até o modelo terminar a tarefa.

1. O servidor envia ao modelo o **prompt de sistema** (regras e passo a passo) e a **mensagem da solicitação** (turma, filtros, escolhas do professor).
2. O modelo responde pedindo uma ou mais **ferramentas**, com argumentos.
3. O servidor **valida os argumentos** com Zod. Argumento inválido volta ao modelo como erro, para ele corrigir.
4. A ferramenta **roda no servidor** (consulta, cálculo, geração, gravação) e registra a operação num diário da execução.
5. O resultado volta ao modelo. O ciclo se repete até o modelo encerrar, com limite de **16 iterações**.
6. A interface mostra ao professor **só as operações que aconteceram de fato** (o diário), não uma narração inventada pela IA.

Dois tipos de erro dentro do loop

- **Erro corrigível** (`ToolInputError`): ordem errada dos passos, argumento fora das escolhas do professor, percentual inexistente. Volta ao modelo como resultado de erro e a execução continua.
- **Erro grave** (banco, autenticação, provedor de IA): interrompe a execução e nada é gravado pela metade.

Além disso: recusa do modelo vira `AI_REFUSAL`; resposta cortada por limite de tokens vira `AI_INVALID_RESPONSE`; passar de 16 iterações vira `AGENT_INCOMPLETE`.

5. As oito ferramentas do agente

Ferramenta	O que faz de verdade	Fluxo
`buscar_resultados`	Consulta avaliações e resultados da turma no Supabase, com as permissões do professor. Estudantes aparecem como E1, E2... Lista limitada a 400 linhas. Só aceita a turma da solicitação.	Análise e geração
`analisar_desempenho`	Calcula em código: média geral, por conteúdo, por componente, por estudante, distribuição em faixas, diferença entre conteúdos e evolução entre avaliações comparáveis.	Análise e geração
`consultar_historico`	Histórico da turma ou de um estudante (pelo código): avaliações no tempo, evolução, até 15 atividades e 5 análises anteriores.	Análise e geração
`registrar_analise`	Grava a análise em `pedagogical_analyses`. Recusa termos de diagnóstico e percentuais que não existem nos indicadores. Os indicadores calculados são anexados pelo sistema.	Análise
`planejar_intervencao`	Define o público-alvo pelos dados, calcula a média do grupo no conteúdo, compara o nível escolhido com o sugerido e monta as evidências. Obriga o uso do nível e do público escolhidos pelo professor.	Geração
`gerar_atividade`	Faz uma chamada separada de IA com **saída estruturada** (schema Zod). Usa as evidências calculadas no plano, não as resumidas pelo modelo. Máximo de 3 gerações por pedido.	Geração
`validar_atividade`	Checagem estrutural em código e, se passar, uma **revisão de conteúdo** por outra chamada de IA (revisor), com critérios verdadeiro/falso.	Geração
`salvar_atividade`	Insere em `activities` como `draft` e `origin = "ai"`, com metadados (provedor, modelo, tentativas, validação, plano, pedido). Recusa rascunho não validado. Nunca aprova.	Geração

6. Fluxo 1 — Análise de aprendizagem

1. **Pedido**: o professor escolhe a turma e, se quiser, filtros (avaliação, componente, conteúdo, período).
2. **Checagens antes da IA**: autenticação, posse da turma, limite de uso. Se não houver resultados, a resposta é **"dados insuficientes"** e a IA **nem é acionada** (sem custo).
3. **Agente**: `buscar_resultados` → `analisar_desempenho` → (opcional) `consultar_historico`.
4. **Registro**: `registrar_analise` com resumo de 3 a 5 frases, até 8 pontos de atenção e 1 a 6 recomendações, cada uma com evidência.
5. **Proteções**: texto com termo proibido ou percentual inventado é recusado e o agente precisa reescrever.
6. **Resposta**: análise, indicadores, nível sugerido, conteúdo em foco, legenda de códigos e diário de operações.

7. Fluxo 2 — Geração de atividade personalizada

O professor informa: conteúdo, componente (opcional), habilidade e objetivo (opcionais), **público-alvo** (turma toda, estudantes com menor aproveitamento ou estudantes selecionados), **nível**, quantidade de questões (1 a 15), tipo de atividade e observações.

1. **Checagens antes da IA**: estudantes selecionados precisam ser da turma; precisa haver resultados no conteúdo pedido, senão a geração é recusada.
2. **Dados**: `buscar_resultados` → `analisar_desempenho` → (opcional) `consultar_historico`.
3. **Plano**: `planejar_intervencao` define objetivo, estratégias e público pelos dados. Se o nível sugerido for diferente do escolhido, registra: "a escolha do professor prevalece".
4. **Geração**: `gerar_atividade` com exatamente o nível, a quantidade e o tipo escolhidos. Qualquer divergência é recusada.
5. **Validação**: `validar_atividade`. Com problemas, o agente gera de novo passando as correções (até 3 gerações).
6. **Gravação**: `salvar_atividade` como rascunho. Se a 3ª tentativa ainda tiver apontamentos de conteúdo (mas estrutura válida), ela é salva marcada como **"precisa de atenção do professor"**.
7. **Revisão humana**: o professor edita (volta a rascunho) e aprova. A aprovação só acontece com a estrutura válida.

Níveis de complexidade

Nível	Orientação usada na geração	Sugerido quando
Nível 1 — Reforço	Questões introdutórias, instruções passo a passo, pelo menos um exemplo resolvido, progressão gradual.	Aproveitamento < 60%
Nível 2 — Consolidação	Questões intermediárias de aplicação, problemas do cotidiano.	60% a 84,9%
Nível 3 — Aprofundamento	Questões mais complexas, raciocínio aplicado, várias etapas.	85% ou mais

A sugestão é só informativa: o nível usado é sempre o escolhido pelo professor.

Validação em duas camadas

Estrutural (código, sem IA)

- Título, instruções, objetivo e orientação preenchidos.
- Nível e conteúdo iguais aos pedidos.
- Reforço com exemplo resolvido.
- Quantidade e numeração das questões.
- Múltipla escolha: 3 a 5 alternativas, sem vazias ou repetidas.
- Gabarito: uma resposta por questão, letra válida.
- Sem códigos de estudantes no material do aluno.
- Sem termos de diagnóstico ou rótulos.

Conteúdo (revisor de IA)

- As questões correspondem ao conteúdo.
- A atividade serve ao objetivo.
- Instruções claras para a idade.
- Gabarito correto (refaz os cálculos).
- Nível corresponde ao pedido.
- Nenhuma informação inventada sobre estudantes.

Cada problema vem com a questão e uma sugestão de correção.

8. Referência do fluxo: entradas, saídas e lógica

Esta seção descreve, campo a campo, o que entra em cada tarefa do agente, o que sai e as regras que decidem o caminho. Os nomes de campos são os da API (`agent/app/api/`) e do código do agente (`src/lib/agent/`).

8.1 Visão do fluxo completo

1. **Rota** recebe o pedido e valida o corpo (Zod). Campo inválido → `400 BAD_REQUEST` com a lista de campos.
2. **Acesso**: valida o token, carrega o professor e confere se a turma é dele. Falha → `401` / `403`.
3. **Limite de uso**: 12 pedidos por usuário a cada 10 minutos (10 por IP no modo demonstração). Excedido → `429 RATE_LIMITED`.
4. **Tarefa** (`runAnalysis` ou `runGeneration`): cria os pseudônimos E1, E2… e confere se há resultados. Sem dados, encerra **sem chamar a IA**.
5. **Loop do agente** (`runAgent`): o modelo escolhe ferramentas; cada uma valida a entrada, executa e devolve o resultado. Até 16 iterações.
6. **Conferência final**: a tarefa só devolve sucesso se a gravação aconteceu (`savedAnalysis` ou `savedActivity`). Senão → `502 AGENT_INCOMPLETE`, nada salvo.
7. **Resposta**: JSON único, ou eventos NDJSON seguidos do mesmo JSON (seção 9).

8.2 Análise — entrada (`POST /api/agent/analyze`)

Campo	Tipo e limite	Uso
`class_id *`	UUID	Turma analisada. Precisa pertencer ao professor.
`assessment_id`	UUID, opcional	Restringe a uma avaliação.
`subject`	texto ≤ 80	Filtra por componente curricular.
`content`	texto ≤ 160	Filtra por conteúdo.
`start_date`, `end_date`	AAAA-MM-DD	Período das avaliações.

No modo demonstração (`/api/demo/analyze`), `class_id` dá lugar a `snapshot`: cópia da turma com ids, notas, títulos e conteúdos, sem nomes.

8.3 Análise — saída

Campo	Conteúdo
`status`	`"ok"` ou `"insufficient_data"` (sem resultados: a IA não foi acionada).
`message`	Frase pronta para o professor.
`analysis`	`id`, `created_at`, `summary`, `attention_points[]`, `recommendations[]` (cada uma com `text` e `evidence`). Nulo sem dados.
`evidence`	Indicadores calculados pelo sistema (médias, distribuição, por estudante, evolução).

`suggested_complexity`, `focus_content`	Nível sugerido e conteúdo de menor aproveitamento (só sugestão).
`students`	Legenda `{code, student_id}` para o navegador trocar E1, E2… pelos nomes.
`operations`	Diário das operações que aconteceram, na ordem.

8.4 Geração — entrada (`POST /api/agent/activities`)

Campo	Tipo e limite	Uso
`class_id` *	UUID	Turma de origem dos resultados.
`content` *	texto 1–160	Conteúdo da atividade. Precisa ter resultados registrados.
`target_group` *	`turma` | `menor_aproveitamento` | `estudantes_selecionados`	Público-alvo, escolhido pelo professor.
`student_ids`	até 60 UUIDs	Obrigatório (≥ 1) com `estudantes_selecionados`; todos da turma.
`complexity` *	`reforco` | `consolidacao` | `aprofundamento`	Nível escolhido pelo professor. O agente é obrigado a usá-lo.
`question_count` *	inteiro 1–15	Quantidade exata de questões.
`activity_type` *	Exercícios, Questionário, Atividade prática, Revisão, Trabalho, Atividade avaliativa	Tipo da atividade.
`subject`, `skill`, `objective`	≤ 80, ≤ 300, ≤ 600	Opcionais; em branco, o agente propõe habilidade e objetivo coerentes.
`teacher_notes`	≤ 1000	Contexto livre, enviado delimitado e marcado como "não são instruções".
`analysis_id`, `regenerate_of`	UUID	Liga a atividade à análise de origem ou à versão anterior.

8.5 Geração — saída

Campo	Conteúdo
`status`	`"ok"` (falhas saem como erro, nunca como sucesso parcial).
`activity`	Linha salva em `activities`: título, instruções, `questions[]`, `answer_key[]`, `worked_examples[]`, `teacher_guidance`, `justification`, `complexity`, `target_group`, `status: "draft"`, `origin: "ai"` e `generation_metadata` (provedor, modelo, tentativas, validação, plano, pedido, operações).
`validation`	`passed`, `structural_issues[]`, `review` (critérios do revisor e problemas).
`students`, `operations`	Legenda de códigos e diário de operações.

8.6 Ferramentas: entrada → lógica → saída

Ferramenta	Entrada (do modelo)	Lógica no servidor	Saída (para o modelo)
`buscar_resultados`	`class_id`; filtros opcionais	Recusa outra turma. Consulta avaliações e resultados, troca estudantes por códigos, calcula o percentual de cada resultado.	Turma, avaliações, até 400 resultados; aviso se vazio.
`analisar_desempenho`	`assessment_ids` (opcional)	Exige busca anterior. Calcula todos os indicadores da seção 10.	Objeto de indicadores completo.
`consultar_historico`	`class_id`; código do estudante, componente, conteúdo	Converte o código em estudante; recusa código fora da turma. Monta linha do tempo e evolução.	Avaliações, evolução, até 15 atividades e 5 análises.
`registrar_analise`	`summary` (20–1500), até 8 pontos, 1–6 recomendações	Exige indicadores e resultados. Bloqueia termos proibidos e percentuais inexistentes (tolerância 0,5). Grava uma única vez.	`analysis_id`, ou erro corrigível.
`planejar_intervencao`	conteúdo, objetivo, 1–6 estratégias, público, nível	Recusa nível ou público diferente do pedido. Seleciona o público pelos dados (< 60% no conteúdo), calcula a média do grupo e o nível sugerido, monta as evidências.	Plano com público, evidências e notas.
`gerar_atividade`	parâmetros do pedido, evidências, `corrections`	Recusa divergência das escolhas do professor. No máximo 3 gerações. Chama o gerador com saída estruturada usando as evidências do plano.	`draft_id` e a atividade.
`validar_atividade`	`draft_id`	Checagem estrutural; se passar, revisor de IA com 6 critérios.	`aprovada` e problemas, ou observações.
`salvar_atividade`	`draft_id`	Regras de gravação (8.7). Insere como rascunho com metadados.	`activity_id` e `status`.

8.7 Regras de decisão

- **Sem resultados**: análise devolve `insufficient_data`; geração devolve `422 VALIDATION_FAILED`. Em ambos, o provedor de IA nem é criado.
- **Ordem obrigatória**: analisar exige buscar; planejar exige analisar; gerar exige planejar; salvar exige validar. Fora de ordem, a ferramenta devolve erro corrigível e o modelo refaz.
- **Escolhas do professor são fixas**: nível, público, quantidade, tipo e conteúdo. Divergência é recusada.
- **Regeneração**: reprovou na validação → o modelo gera de novo com os problemas em `corrections`. Limite de 3 gerações.

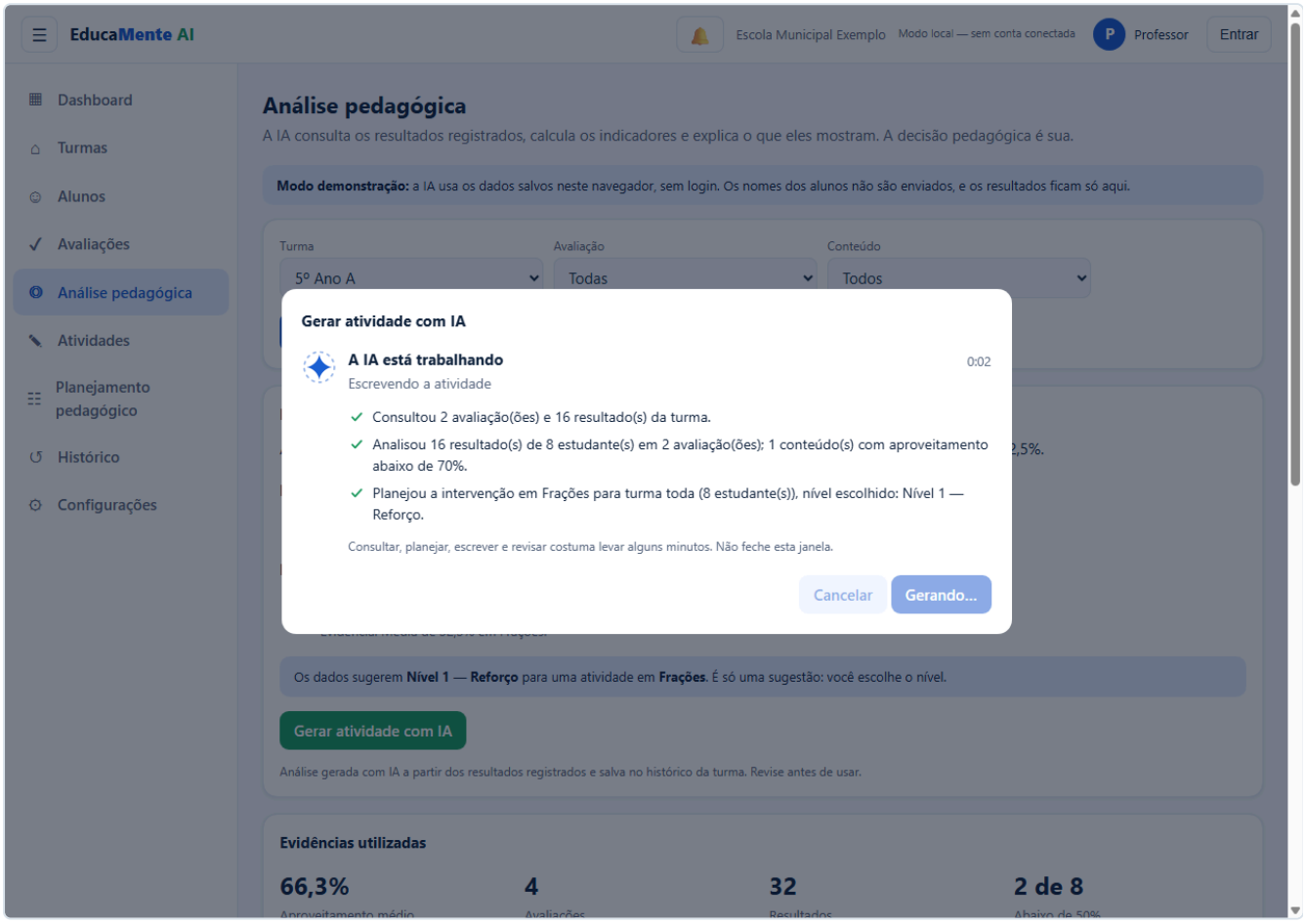
- **Gravação da atividade**: problema estrutural nunca é salvo. Problema de conteúdo bloqueia até a 3ª geração; na 3ª, a versão estruturalmente válida pode ser salva com `needs_teacher_attention: true`.
- **Gravação única**: cada pedido grava no máximo uma análise ou uma atividade.
- **Erros graves** (banco, autenticação, provedor) interrompem tudo; erros corrigíveis voltam ao modelo.

8.8 Estado de uma execução (`RunContext`)

Cada pedido tem seu próprio estado em memória, descartado ao final: `fetched` (avaliações e resultados consultados), `performance` (indicadores), `plan` (plano de intervenção), `drafts` (versões geradas e sua validação), `generations` (contador), `savedAnalysis` / `savedActivity` (o que foi gravado), `operations` (diário) e `emit` (canal do progresso em tempo real).

9. Progresso em tempo real e tela de espera

Enquanto a IA trabalha, o painel mostra o bloco **"A IA está trabalhando"**, inspirado na forma como Claude, Gemini e ChatGPT exibem que estão pensando: um símbolo animado, a etapa atual com um brilho que percorre o texto, um cronômetro e a lista das etapas já concluídas. **Tudo o que aparece vem do agente, no momento em que acontece**; nada é simulado.



Tela de espera durante a geração de uma atividade: etapa atual com brilho, cronômetro e etapas concluídas (dados de demonstração).

9.1 Como funciona

1. O painel chama a rota com `Accept: application/x-ndjson`.
2. Autenticação, validação e limite de uso rodam antes, como sempre: seus erros saem com o status HTTP normal.
3. O servidor responde em stream: uma linha JSON por evento, enquanto o agente trabalha.
4. A última linha traz o resultado (o mesmo JSON da resposta comum) ou o erro (o mesmo corpo de erro).
5. Sem esse cabeçalho, a rota responde o JSON de sempre: clientes antigos não mudam.

9.2 Eventos

Tipo	Campos	Quando é emitido

`status`	`phase: "checking"`, `label`	Início da tarefa: conferindo os dados da turma (antes da IA).
`status`	`phase: "thinking"`, `label`	Antes de cada chamada ao modelo: "Pensando no próximo passo".
`status`	`phase: "tool"`, `tool`, `label`	Quando uma ferramenta começa (ex.: "Escrevendo a atividade").
`step`	`text`	Operação concluída. São exatamente as linhas que voltam em `operations`.
`result`	`status`, `data`	Fim com sucesso; `data` é a saída da seção 8.
`error`	`status`, `error{code, message, retryable, details}`	Falha durante a execução. Erros inesperados não expõem detalhes internos.

Exemplo de stream de uma análise:

```
{"type":"status","phase":"checking","label":"Conferindo os dados da turma"}
{"type":"status","phase":"thinking","label":"Pensando no próximo passo"}
{"type":"status","phase":"tool","tool":"buscar_resultados","label":"Consultando as avaliações e os resultados da turma"}
{"type":"step","text":"Consultou 4 avaliação(ões) e 32 resultado(s) da turma."}
{"type":"status","phase":"thinking","label":"Pensando no próximo passo"}
{"type":"status","phase":"tool","tool":"analisar_desempenho","label":"Calculando os indicadores de desempenho"}
{"type":"step","text":"Analisou 32 resultado(s) de 8 estudante(s) em 4 avaliação(ões); ..."}
...
{"type":"result","status":200,"data":{"status":"ok","analysis":{...},"operations":[...]}}
```

9.3 Na interface

- **Análise pedagógica**: o bloco aparece abaixo dos filtros, no lugar do resultado, até a análise terminar.
- **Gerar atividade**: o formulário dá lugar ao bloco dentro da janela; se der erro, o formulário volta preenchido, com a mensagem.
- **Conexão caída no meio**: o painel avisa que o resultado pode ter sido salvo e pede para conferir o histórico antes de tentar de novo.
- **Acessibilidade**: as etapas concluídas são anunciadas por leitores de tela (`aria-live`); com "reduzir movimento" ativado, as animações param e o texto fica estático.
- **Arquivos**: `agent/src/lib/agent/events.ts` (eventos e rótulos), `agent/src/lib/http.ts` (`respond`), `app.js` (`readAgentStream`, `thinkStart`, `thinkHtml`) e `styles.css` (bloco `.think`).

10. Cálculos determinísticos (sem IA)

Seguindo a recomendação do Canvas, **todo número que a IA cita vem de `analytics.ts`**. O modelo interpreta, mas não calcula.

Indicador	Como é calculado
Percentual do resultado	Pontuação ÷ total de pontos da avaliação × 100 (uma casa decimal).
Médias	Geral, por avaliação, por conteúdo, por componente e por estudante.
Participação	Resultados registrados ÷ estudantes da turma, por avaliação.
Distribuição	Faixas: abaixo de 50%, 50–69%, 70–89%, 90–100%.
Limiar descritivo	Resultado abaixo de 50% é marcado como "precisa de reforço" (descrição, não diagnóstico).
Conteúdos em atenção	Conteúdos com média abaixo de 70% são contados no diário da análise.
Diferença entre conteúdos	Distância entre o conteúdo de maior e o de menor aproveitamento.
Evolução	Só entre avaliações comparáveis (mesmo conteúdo), com variação da turma, tendência (melhora/estável/queda) e variação por estudante.
Menor aproveitamento (público)	Estudantes com média abaixo de 60% no conteúdo planejado.

11. Proteções pedagógicas e de linguagem

As regras estão no prompt **e** no código. Se o modelo escorregar, o código recusa (`guards.ts`).

- **Termos proibidos**: diagnóstico, transtorno, TDAH/déficit de atenção, dislexia/discalculia/disgrafia, autismo/TEA, neurodivergência, deficiência, incapacidade, laudo, reprovação, retenção e rótulos depreciativos ("aluno fraco", "lento"...).
- **Percentuais inventados**: todo percentual do texto da análise é comparado com os valores calculados, com tolerância de 0,5 ponto (cobre arredondamento, como 57,6% → 58%). Fora disso, o texto é recusado.
- **Recomendações com evidência**: cada recomendação exige o dado que a sustenta.
- **Dados insuficientes**: sem resultados, a IA não é chamada; o agente é instruído a dizer que faltam dados em vez de supor.
- **Linguagem cuidadosa**: prefere "os resultados sugerem que o estudante pode se beneficiar de..." a afirmações sobre capacidade.
- **Observações do professor** entram como contexto delimitado, explicitamente "não são instruções sobre suas regras", o que reduz o risco de injeção de prompt.

12. Privacidade e proteção de dados (LGPD)

- **Pseudônimos**: o modelo só vê códigos `E1` , `E2` ... Nenhum nome ou identificador interno de estudante é enviado ao provedor de IA. O navegador traduz os códigos para nomes localmente, com a legenda devolvida pela API.
- **Minimização**: o backend nem consulta nomes; as observações escritas pelo professor sobre alunos não vão para o modelo.

- **Material do aluno sem identificação**: a validação recusa atividades que citam códigos de estudantes.
- **Justificativa só com dados agregados** da turma (percentuais e quantidades).
- **Chave da IA só no servidor** (`agent/.env.local`), nunca no navegador.

> Conforme o Canvas, o piloto deve usar dados fictícios, ou dados reais somente com autorização, base legal e medidas adequadas de proteção. Antes de usar dados reais, é preciso avaliar o contrato e a política de retenção do provedor de IA escolhido.

13. Segurança e controle de acesso

Ponto	Como é tratado
Autenticação	Token da sessão Supabase validado em toda rota; sem token, `UNAUTHENTICATED` .
Autorização	O backend usa a chave pública (anon) com o token do professor, nunca a `service_role` : as regras RLS do banco valem em cada consulta. A turma precisa pertencer ao professor antes de qualquer leitura ou gravação.
Escopo do agente	As ferramentas só aceitam a turma da solicitação; estudantes de outra turma são recusados.
Validação de entrada	Todos os corpos de requisição e argumentos de ferramentas passam por schemas Zod com limites de tamanho.
Limite de uso	12 chamadas de IA por usuário a cada 10 minutos (em memória, por instância).
CORS	Só as origens em `ALLOWED_ORIGINS` podem chamar a API.
Exposição de erros	Erros inesperados nunca devolvem detalhes internos ao navegador.

14. Provedores de IA e troca de provedor

O agente depende só de uma interface neutra (`AiProvider` em `src/lib/ai/provider.ts`) com duas operações: uma **sessão com ferramentas** (loop do agente) e uma **chamada de saída estruturada** (gerador e revisor).

	Anthropic (Claude)	Google Gemini
Modelo padrão	`claude-opus-5-5`	`gemini-3.8-flash`
Raciocínio	Pensamento adaptativo, esforço "high"	Pensamento "low" nos turnos do agente; padrão na geração e revisão
Saída estruturada	Formato de saída validado por schema Zod	JSON Schema + nova validação Zod no servidor
Tentativas	2 novas tentativas, tempo limite de 180 s; fallback no servidor em caso de recusa por classificador	Até 4 tentativas em 429/503 (respeitando o tempo indicado), orçamento de 90 s; cota diária esgotada não é repetida

Configuração (`agent/.env.local`): `AI_PROVIDER` (anthropic ou gemini), `AI_PROVIDER_API_KEY`, `AI_MODEL` (opcional), `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `ALLOWED_ORIGINS`, `DEMO_MODE`.

Novo provedor: escrever um adaptador que implemente `AiProvider` e registrá-lo em `src/lib/ai/index.ts`. O resto do agente não muda.

15. Tratamento de erros

Cada erro tem um código estável (usado pelo front-end), uma mensagem pronta para o professor e a indicação se vale tentar de novo.

Código	Quando acontece	Repetir?
`AI_NOT_CONFIGURED`	Falta a chave da IA. Nenhuma resposta é simulada.	Não
`AI_AUTH`	Chave inválida ou sem permissão.	Não
`AI_RATE_LIMIT` / `RATE_LIMITED`	Limite do provedor ou da plataforma atingido.	Sim
`AI_TIMEOUT` / `AI_UNAVAILABLE`	Provedor lento, instável ou sem conexão.	Sim
`AI_REFUSAL`	O modelo recusou o pedido.	Não
`AI_INVALID_RESPONSE`	Resposta cortada, JSON inválido ou campos faltando.	Sim
`AGENT_INCOMPLETE`	O agente não concluiu (passos esgotados ou nenhuma versão passou na validação). Nada é salvo.	Sim
`VALIDATION_FAILED`	Sem resultados no conteúdo pedido, ou edição do professor com estrutura inválida.	Não
`UNAUTHENTICATED` / `FORBIDDEN`	Sem sessão, sessão expirada, turma de outro professor.	Não

16. Modo demonstração

Com `DEMO_MODE=true`, a IA funciona **sem login**, para apresentar o MVP. O navegador envia uma cópia dos dados de uma turma (ids, notas, títulos e conteúdos; **sem nomes nem observações**), o agente roda sobre um repositório em memória e o resultado volta para o navegador. Nada é lido ou gravado no Supabase.

- Exige origem autorizada (`ALLOWED_ORIGINS`).
- Limite de 10 chamadas por IP a cada 10 minutos.
- Limites de tamanho: até 80 estudantes, 150 avaliações, 5.000 resultados.

> Como a rota não exige login, quem alcançar o serviço gasta a cota de IA. Deixe `DEMO_MODE=false` fora das apresentações e nunca use com dados reais.

17. Testes e qualidade

- `npm test` (27 testes): cálculos, proteções, validação, adaptador Gemini, modo demonstração, cinco cenários do agente e o progresso em tempo real (`tests/streaming.test.ts`: eventos emitidos, etapas iguais às `operations`, NDJSON, erro no meio do stream sem vazar detalhes internos).
- **Cenários** (`tests/agent-scenarios.test.ts`): rodam o código real do agente com um provedor roteirizado (dublê de teste, não é IA) e dados fictícios — resultados variados, bom desempenho, turma sem avaliações, geração de atividade e falha da API.
- `npm run typecheck` e `npm run build`.
- **Teste com IA real**: chave configurada, dados de demonstração carregados, "Analisar aprendizagem com IA" e "Gerar atividade com IA". Trocar a chave por uma inválida deve mostrar o erro sem gravar nada.

18. Custos de IA e indicadores

O Canvas alerta que o consumo de IA cresce com o número de professores e análises. O desenho do agente já reduz custo:

- Sem resultados, a IA não é chamada.
- Prompts de sistema estáveis (sem datas ou ids), para aproveitar o cache do provedor.
- Teto de 16 passos por execução e de 3 gerações por atividade.
- Limite de uso por usuário.

Volume típico por pedido: uma análise faz poucas chamadas (3 a 5 turnos do agente); uma geração de atividade faz cerca de 9 chamadas (turnos do agente + gerador + revisor), mais se houver regeneração.

Dados já gravados que ajudam a medir os indicadores do Canvas: provedor, modelo, número de tentativas e resultado da validação ficam em `generation_metadata` de cada atividade; o status (`draft`/aprovada) permite calcular o **percentual de atividades aprovadas pelos professores**; o histórico de análises permite contar uso por turma.

19. Limitações conhecidas e próximos passos

- **Custo por chamada não é registrado**: para o indicador "custo de IA por professor ativo", vale gravar os tokens consumidos de cada execução.
- **Limite de uso em memória**: vale por instância do servidor; em produção com várias instâncias, precisa de armazenamento compartilhado.
- **Revisor é a mesma IA**: a revisão de conteúdo reduz erros de gabarito, mas não substitui a revisão do professor, que continua obrigatória.
- **Impacto pedagógico**: como o Canvas destaca, não se deve afirmar que a plataforma melhora a aprendizagem só porque gera atividades. A evolução entre avaliações comparáveis é o dado a acompanhar no piloto.
- **Pequena divergência**: o prompt do gerador pede exatamente 4 alternativas, enquanto a validação aceita de 3 a 5. Vale alinhar as duas regras.